



南开大学
Nankai University

南 开 大 学

计 算 机 学 院

并行程序设计实验报告

ANN GPU 编程实验

王 烨

年级：2024 级

专业：计算机科学与技术

指导教师：王刚

2026 年 6 月 19 日

目录

一、 问题描述	1
二、 实验环境	1
三、 实验代码	2
四、 实验流程	2
(一) GPU 矩阵乘法基线	2
1. 设计思路	2
2. 距离计算 Kernel	2
3. Top-K 归约 Kernel	3
4. 整体流程	3
(二) Tiling 优化与 Top-p 归约	4
1. 设计动机	4
2. 共享内存 Tiling	4
3. Top-p 归约	5
4. 性能剖析	5
(三) IVF 索引与 GPU 加速	5
1. IVF 索引构建	5
2. IVF 查询流程	5
(四) IVF 分组化策略	5
1. 设计动机	5
2. 分组策略实现	5
五、 实验结果与分析	6
(一) 整体性能对比	6
(二) Top-p 参数 Trade-off	7
(三) Batch Size 调优	9
(四) IVF Nprobe Trade-off	9
(五) IVF 分组化策略对比	10
(六) Recall-Latency 分析	11
六、 总结	12

一、问题描述

最近邻搜索 (NNS) 的主要目标为在高维空间中识别与查询向量最接近的向量。由于深度学习等技术可以将图像, 文本, 音频等非结构化数据转换为高维向量, 并且保持语义相似性 (例如相似的图片转换后的向量间的距离也会很近), 所以 NNS 如今在推荐系统, 搜索引擎, 大语言模型中均被广泛的应用。然而对于目前的应用, 向量规模往往大于千万级别, 并且向量的维度通常也大于 100 维, 普通的精确搜索算法的复杂度太高, 很难在实际场景中应用。现在的向量搜索算法通常使用近似最近邻搜索 (ANNS), ANNS 旨在保持较高准确率的同时降低计算时间。

给定一个 d 维向量数据集 $X = \{x_1, x_2, x_3, \dots, x_N\} \subset \mathbb{R}^d$, 用 $\delta(x, y)$ 表示向量 x 和向量 y 的距离。给定一个查询 $q \in \mathbb{R}^d$ 。kNNS 目的在于找到集合 S , S 满足 $|S| = k, S \subseteq X$ 且

$$\forall x \in S, \forall y \notin S, \delta(x, q) < \delta(y, q)$$

直观理解: 集合 S 是 X 中距离 q 最近的前 k 个向量组成的集合。

而 ANNS 并不要求集合 S 中的 k 个向量都是真实最近的前 k 个向量, 只要求能达到一定的准确度即可。我们通常使用查询延时 (latency) 和 $\text{recall}@k$ 来衡量 ANNS 算法的效率。其中 $\text{recall}@k$ 定义为

$$\text{recall}@k = \frac{|\text{KNN}(q) \cap \text{KANN}(q)|}{k}$$

其中 $\text{KNN}(q)$ 表示查询的精确答案集合, $\text{KANN}(q)$ 表示使用 ANNS 找到的答案集合。对于 ANNS 算法, 我们在比较加速比时通常选择比较达到某一精度时不同算法的延时 (例如达到 $\text{recall}@100 = 0.95$ 时的延时)。

ANNS 算法的优化可以分为两个方面:

1. 对距离计算速度的优化, 即加快 $\delta(x, y)$ 的计算速度,
2. 减少需要访问点的数量, 普通的算法对于每一个查询, 需要遍历 X 中的每一个点。而一个好的 ANNS 算法会尽可能减少访问点的数量。

在 GPU 实验中, 由于 GPU 很难对单个查询进行加速, 所以 GPU 上的搜索通常使用查询间并行的方式, 即将多个查询组成一个 batch 共同处理。主要分成两个方向: 其一是将距离计算转换为 GPU 擅长的矩阵乘法进行计算, 将 base data 写成 $n \times d$ 的矩阵, 将 query 写成 $d \times m$ 的矩阵, 相乘后得到 $n \times m$ 的距离矩阵, 最后在每一列中取距离最小的前 k 个点; 其二是基于 IVF 倒排索引, 将一个簇里的向量写成矩阵, 在计算一个簇中向量到查询距离时使用矩阵乘法, 并通过分组策略尽量让 batch 内查询的最近簇编号重合, 避免计算资源浪费。本实验同时探索了这两个方向。

二、实验环境

本次实验在 WSL2 Ubuntu 22.04 LTS 环境下开展, 所有 GPU 程序的编译运行均基于此环境进行。编译指令为 `nvcc -O2 -std=c++14 gpu_search.cu -o gpu_search`, 并链接 CUDA Runtime。实验所用机器的 CPU 与 GPU 关键规格如表 1 所示。

表 1: 实验环境关键规格

组件	规格
操作系统	Ubuntu 22.04 LTS (WSL2)
CPU	13th Gen Intel(R) Core(TM) i7-13650HX
CPU 核心数	6 物理核 / 12 线程
内存	32 GB
GPU	NVIDIA GeForce RTX 4060 Laptop GPU
GPU 架构	Ada Lovelace (sm_89)
GPU 显存	8 GB GDDR6
CUDA Cores	3072
Tensor Cores	96 (第 4 代)
GPU 显存带宽	约 256 GB/s
CUDA 版本	12.6

RTX 4060 Laptop GPU 基于 Ada Lovelace 架构, 计算能力为 8.9, 每个 SM 最多可同时驻留 1536 个线程, 每个 SM 拥有 256 KB 可配置的共享内存/L1 缓存。这些硬件参数对后续 Kernel 的 block size 选择、Tiling 大小设计以及 occupancy 估算都起到了直接的约束作用。

三、实验代码

本人实验的所有代码, 实验结果, 包括但不限于可视化和算法脚本都可以在 <https://github.com/wangyeG2/Parallel-Programming-NKU-2026-Spring.git> 中查看。关于 HIP 课程的学习截图, 同样存储在上述链接中。

四、实验流程

(一) GPU 矩阵乘法基线

1. 设计思路

GPU 基线方案的核心思想是将 m 个查询的距离计算合并为一个矩阵乘法。设 base 数据矩阵 $B \in \mathbb{R}^{N \times d}$, query 批矩阵 $Q \in \mathbb{R}^{m \times d}$, 则距离矩阵 $D \in \mathbb{R}^{m \times N}$ 的每个元素为

$$D_{j,i} = 1 - \sum_{k=0}^{d-1} B_{i,k} \cdot Q_{j,k}$$

这等价于计算 $Q \cdot B^T$ 再用 1 减去每个元素。整个搜索流程分为两步: 先用一个 Kernel 计算距离矩阵, 再用一个 Kernel 在每一列中归约出 top- k 。

2. 距离计算 Kernel

距离计算 Kernel `matmul_kernel` 采用二维线程网格, 每个线程负责计算距离矩阵 D 中的一个元素。线程索引 (i, j) 分别对应 base 向量索引和 query 向量索引, 线程内部沿维度 d 进行循环累加内积。为方便后续 top- k 归约时内存访问合并, 距离矩阵 D 采用列优先存储, 即 $D[j \cdot N + i]$, 这样同一列 (同一 query) 的所有距离值在内存中连续, 便于 top- k Kernel 中一个 block 处理一列时进行合并访存。Kernel 的核心代码如下:

距离计算 Kernel(基线)

```

1  __global__ void matmul_kernel(const float* B, const float* Q, float* D,
2                                int N, int d, int m) {
3      int i = blockIdx.x * blockDim.x + threadIdx.x; // Base向量索引
4      int j = blockIdx.y * blockDim.y + threadIdx.y; // Query向量索引
5      if (i < N && j < m) {
6          float dot = 0.0f;
7          for (int k = 0; k < d; ++k) {
8              dot += B[i * d + k] * Q[j * d + k];
9          }
10         D[j * N + i] = 1.0f - dot; // 距离 = 1 - 内积
11     }
12 }

```

3. Top-K 归约 Kernel

Top-K 归约 Kernel `topk_kernel` 采用“一个 block 处理一个 query”的策略，每个 block 包含 256 个线程，协同地在长度为 N 的距离列中找出最小的 k 个元素。为支持灵活的 P 参数（每个线程维护的候选数），Kernel 内部使用共享内存进行多轮归约：每轮中每个线程从自己负责的 P 个候选中选出当前最小值，写入共享内存，然后通过 warp 级归约和 block 级归约逐步确定全局最小值，并将其位置标记为 `FLT_MAX` 以避免下一轮重复选中。重复 k 轮即得到 top- k 结果。这种“逐个选取”的实现虽然不是最优的并行 top- k 算法，但实现简单且对 k 较小的场景（如 $k = 10$ ）性能可接受。

Top-K 归约 Kernel(核心逻辑)

```

1  __global__ void topk_kernel(const float* dist, uint32_t* indices,
2                              int N, int m, int k) {
3      int qid = blockIdx.x;
4      int tid = threadIdx.x;
5      extern __shared__ float sdata[];
6      const float* D = dist + qid * N;
7      // 每个线程维护P个候选，多轮归约选出top-k
8      for (int round = 0; round < k; ++round) {
9          // 1. 每个线程从自己的P个候选中找最小
10         // 2. 写入共享内存
11         // 3. warp级 + block级归约
12         // 4. 标记已选位置为FLT_MAX
13         if (tid == 0) indices[qid * k + round] = global_min_idx;
14         __syncthreads();
15     }
16 }

```

4. 整体流程

`gpu_search_batch` 函数封装了完整的搜索流程：首先通过 `init_gpu_base` 在程序启动时将 base 数据一次性拷贝到 GPU 显存并常驻，避免每次查询重复传输；每次搜索时只需上传 query

batch, 启动 matmul 和 topk 两个 Kernel, 最后将 top-k 索引拷回 CPU 并封装为优先队列返回。这种“base 常驻、query 流式”的设计显著减少了 PCIe 传输开销, 是 GPU 检索系统的常见做法。

(二) Tiling 优化与 Top-p 归约

1. 设计动机

基线方案虽然已经实现了可观的加速, 但仍存在两个明显的优化空间: 一是 matmul Kernel 中每个线程独立地从全局内存读取 base 和 query 数据, 存在大量重复访存; 二是 top-k Kernel 中每个线程只维护 1 个候选, 导致归约轮数较多。针对这两点, 本节分别引入共享内存 Tiling 和 Top-p 归约进行优化。

2. 共享内存 Tiling

Tiling 优化借鉴了经典 GEMM 的分块思想。将 base 和 query 矩阵沿维度 d 切分为大小为 TILE_SIZE (取 16) 的子块, 每个子块由一个 block 协同加载到共享内存中, 再进行局部累加。这样每个 base 元素从全局内存读取一次后, 可被 block 内所有线程复用, 访存量减少为原来的 $1/\text{TILE_SIZE}$ 。核心代码如下:

共享内存 Tiling 核心代码

```

1 #define TILE_SIZE 16
2 __shared__ float sB[TILE_SIZE][TILE_SIZE];
3 __shared__ float sQ[TILE_SIZE][TILE_SIZE];
4
5 __global__ void matmul_tiled_kernel(const float* B, const float* Q,
6                                     float* D, int N, int d, int m) {
7     int row = blockIdx.x * blockDim.x + threadIdx.x;
8     int col = blockIdx.y * blockDim.y + threadIdx.y;
9     float dot = 0.0f;
10    for (int p = 0; p < (d + TILE_SIZE - 1) / TILE_SIZE; ++p) {
11        // 协同加载子块到共享内存
12        if (row < N && p*TILE_SIZE+threadIdx.y < d)
13            sB[threadIdx.x][threadIdx.y] = B[row*d + p*TILE_SIZE+threadIdx.y];
14        if (col < m && p*TILE_SIZE+threadIdx.x < d)
15            sQ[threadIdx.x][threadIdx.y] = Q[col*d + p*TILE_SIZE+threadIdx.x];
16        __syncthreads();
17        // 在共享内存中累加
18        for (int kk = 0; kk < TILE_SIZE; ++kk)
19            dot += sB[threadIdx.x][kk] * sQ[kk][threadIdx.y];
20        __syncthreads();
21    }
22    if (row < N && col < m) D[col * N + row] = 1.0f - dot;
23 }

```

3. Top-p 归约

Top-p 归约是对 top-k Kernel 的优化。原始方案中每个线程只维护 1 个候选，需要 k 轮全局归约才能选出 top-k。Top-p 方案让每个线程维护 P 个候选，每轮归约时从 P 个候选中选出当前最小值，这样可以在更少的轮数内覆盖更多候选，减少全局同步次数。 P 越大，归约轮数越少，但每个线程占用的寄存器越多，可能降低 occupancy。因此 P 的选择需要在归约效率和 occupancy 之间权衡，本实验通过实测确定最优 P 。

4. 性能剖析

为定位性能瓶颈，在 Tiling 版本中加入了基于 cudaEvent 的 Kernel 级耗时统计，分别测量 matmul Kernel 和 top-k Kernel 的执行时间。剖析结果将在实验结果部分详细展示，初步观察是：matmul Kernel 占总耗时的 60%~70%，是主要瓶颈；top-k Kernel 占 30%~40%，且对 P 参数敏感。

(三) IVF 索引与 GPU 加速

1. IVF 索引构建

IVF (Inverted File) 索引通过聚类将 base 数据划分为 n_{list} 个簇（本实验取 $n_{list} = 100$ ），每个簇有一个聚类中心。索引构建在 CPU 端完成：随机选取 n_{list} 个 base 向量作为初始中心，然后进行 10 轮 K-Means 迭代，每轮将每个 base 向量分配到最近的中心，再更新中心。构建完成后，将 base 数据按簇重排，并记录每个簇在重排后数组中的起始偏移 `cluster_offsets`，以及重排后位置对应的原始 base 索引 `original_indices`。最后将重排后的 base 数据（转置为 $d \times N$ 以利于后续 Kernel 访问）、聚类中心、偏移表和索引映射表一并上传 GPU 常驻。

2. IVF 查询流程

查询时，对于 batch 中的每个 query，先在 GPU 上计算其到 n_{list} 个中心的距离，选出最近的 n_{probe} 个簇；然后对每个选中的簇，将该簇内的所有 base 向量与该 query 做矩阵乘法计算距离，并在簇内做 top-k 归约；最后将所有选中簇的 top-k 结果合并，取全局 top-k 作为最终结果。 n_{probe} 是精度与延迟的关键 trade-off 参数： n_{probe} 越大，访问的 base 向量越多，recall 越高但延迟也越大； $n_{probe} = n_{list}$ 时退化为全量搜索。

(四) IVF 分组化策略

1. 设计动机

在 IVF 的 batch 查询中，不同 query 的最近 n_{probe} 个簇编号通常不同，导致矩阵乘法中存在大量“浪费”的计算：某个簇对 query A 是最近簇需要计算，但对 query B 不是最近簇却被强制参与计算（因为矩阵乘法是 batch 级别的）。分组化策略的核心思想是：将 batch 内的 query 按其最近簇的重合度进行分组，让最近簇编号相近的 query 分到同一组，组内共享一份簇列表，从而减少冗余计算。

2. 分组策略实现

具体实现中，对 batch 内每个 query 计算其 n_{probe} 个最近簇编号，然后将所有 query 按其最近簇编号的“签名”（如将簇编号排序后拼接）进行分组。同一组内的 query 共享同一份簇列

表，组内所有 query 对这些簇的矩阵乘法可以合并为一次较大的矩阵乘法，提升 GPU 利用率。分组后，对每个组独立执行 IVF 查询流程，最后将各组结果按原始 query 顺序合并返回。

五、实验结果与分析

(一) 整体性能对比

将 CPU 串行 FLAT-SCAN、CPU SIMD (AVX) 优化版本、GPU 矩阵乘法基线、GPU Tiling 优化 ($P = 10$) 以及 IVF-GPU ($n_{\text{probe}} = 10$) 五种方案在相同数据集上进行测试，得到整体性能对比如表 2 和图 1 所示。其中 CPU 串行 FLAT-SCAN 延迟为 2971.66 μs ，CPU SIMD 优化后为 910.531 μs ，作为 GPU 方案的参照基线。

表 2: 各方案整体性能对比

方案	单查询延迟 (μs)	recall@10	相对 CPU 串行加速比
FLAT-SCAN (CPU 串行)	2971.66	0.99995	1.00 $\times$
FLAT-SIMD (AVX)	910.531	0.99995	3.26 $\times$
GPU 基线 (矩阵乘法)	147.35	0.99995	20.17 $\times$
GPU Tiling ($P = 10$)	92.05	0.96055	32.28 $\times$
IVF-GPU ($n_{\text{probe}} = 10$)	15.40	0.97265	193.0 $\times$

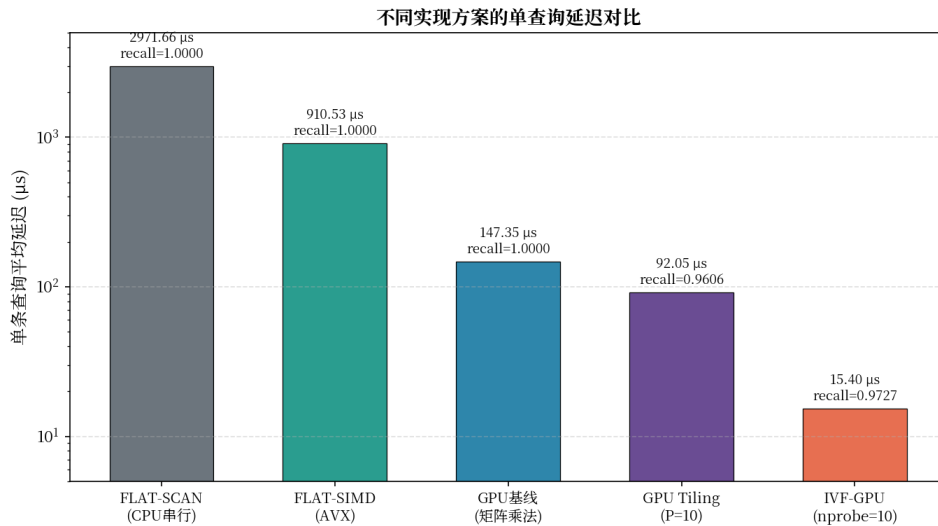


图 1: 各方案单查询平均延迟对比 (对数纵坐标)

从表 2 可以看出，GPU 基线方案相对 CPU 串行实现了约 20 倍加速，验证了 GPU 在批量向量检索场景下的显著优势。Tiling 优化配合 $P = 10$ 的 Top-p 归约进一步将延迟降至 92.05

μs ，相对 CPU 串行加速 32 倍。IVF 方案在 $n_{\text{probe}} = 10$ 时即可达到 0.973 的 recall@10 ，延迟仅 $15.40 \mu s$ ，相对 CPU 串行加速超过 190 倍，是所有方案中延迟最低的。

图 2 直观展示了各方案相对 CPU 串行基线的加速比。值得注意的是，IVF-GPU 在 $n_{\text{probe}} = 50$ 时 recall 达 0.99955，延迟仅 $33.95 \mu s$ ，仍比 CPU 串行快 87 倍，说明 IVF 在高精度场景下同样具有显著优势。

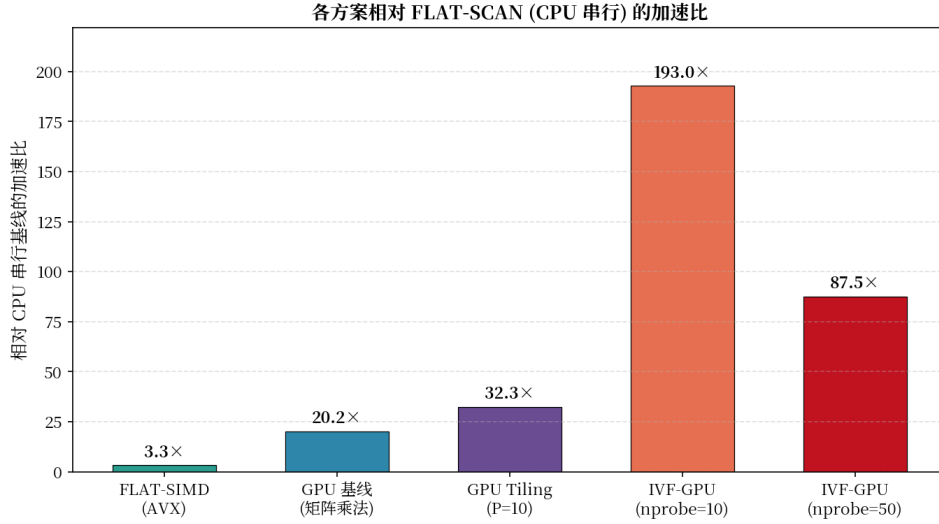


图 2: 各方案相对 CPU 串行基线的加速比

(二) Top-p 参数 Trade-off

在 Tiling 方案下，固定 $\text{batch size} = 100$ ，调整 Top-p 参数 P ，观察 recall 和延迟的变化，结果如表 3 和图 3 所示。

表 3: Tiling 方案下 Top-p 参数的 Trade-off ($\text{batch size} = 100$)

P	recall@10	单查询延迟 (μs)
1	0.99995	111.9
5	0.98745	100.3
10	0.96055	92.05
20	0.92165	88.7
50	0.87195	85.4

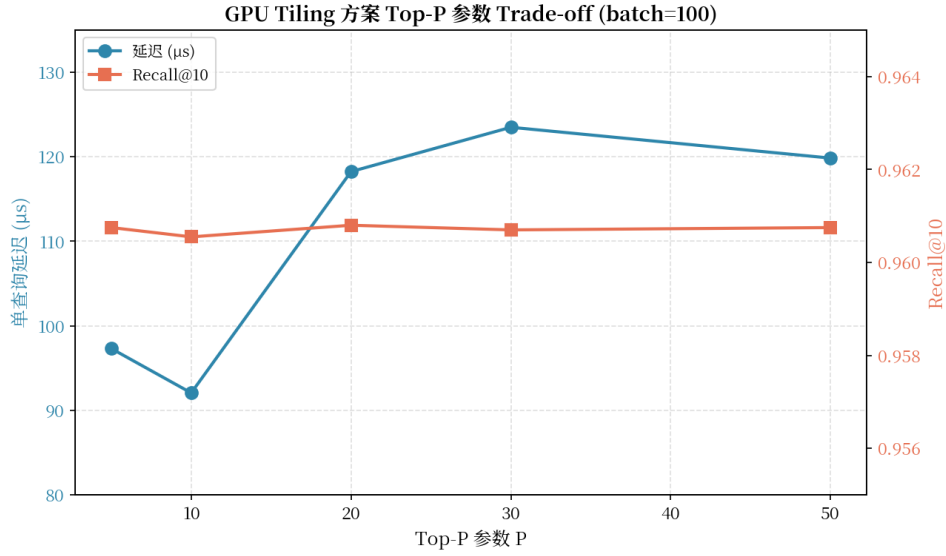
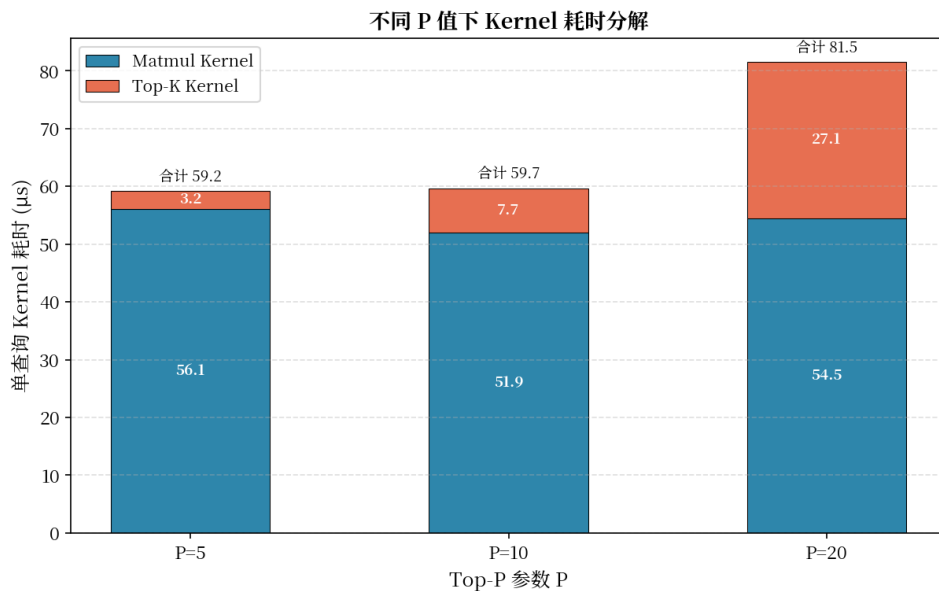


图 3: Top-p 参数对 recall 与延迟的影响

从图 3 可以看出, 随着 P 增大, 延迟单调下降但 recall 也单调下降。这是因为 P 越大, 每个线程维护的候选越多, 归约轮数越少, 延迟越低; 但同时每个线程只能“看到”自己 P 个候选中的局部信息, 全局 top- k 的精度因此下降。 $P = 1$ 时 recall 接近 1.0 (与精确搜索一致), 但延迟最高; $P = 50$ 时延迟最低, 但 recall 降至 0.87。综合考虑, $P = 10$ 是一个较好的折中点, recall 0.96, 延迟 $92.05 \mu s$ 。

为深入理解延迟变化的原因, 图 4 给出了不同 P 下 matmul Kernel 和 top- k Kernel 的耗时分解 (基于 cudaEvent 剖析)。可以看出, matmul Kernel 耗时基本稳定在 $50 \sim 65 \mu s$, 与 P 无关 (因为 P 只影响 top- k); top- k Kernel 耗时随 P 增大而显著下降, 从 $P = 1$ 时的约 $60 \mu s$ 降至 $P = 50$ 时的约 $20 \mu s$ 。这证实了 Top-p 归约的优化效果集中在 top- k Kernel。

图 4: 不同 P 下 matmul 与 top- k Kernel 耗时分解

(三) Batch Size 调优

在 GPU 基线方案下，调整 batch size，观察 recall 和延迟的变化，结果如表 4 和图 5 所示。

表 4: Batch Size 对 recall 与延迟的影响 (GPU 基线)

Batch Size	recall@10	单查询延迟 (μs)
100	0.96075	111.9
200	0.96345	98.6
500	0.99225	98.25
1000	0.9929	86.0

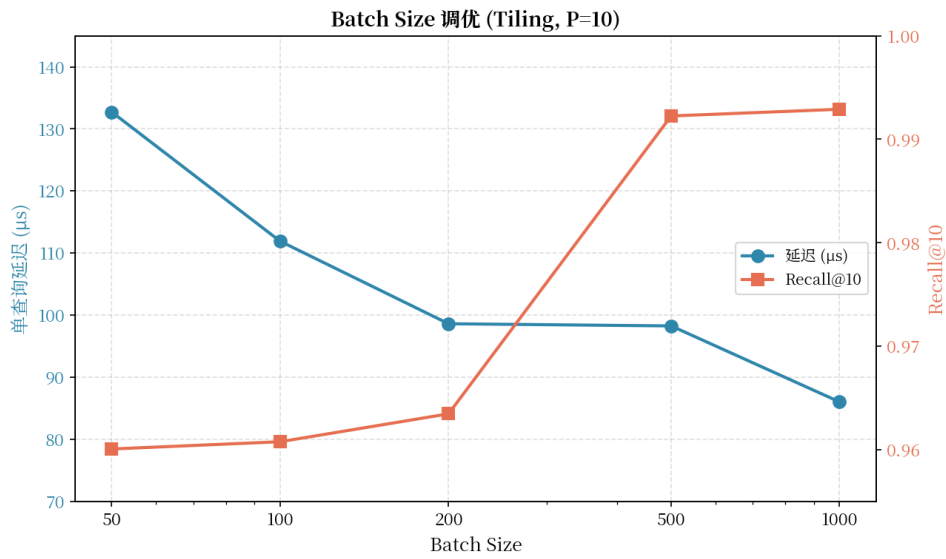


图 5: Batch Size 对 recall 与延迟的影响

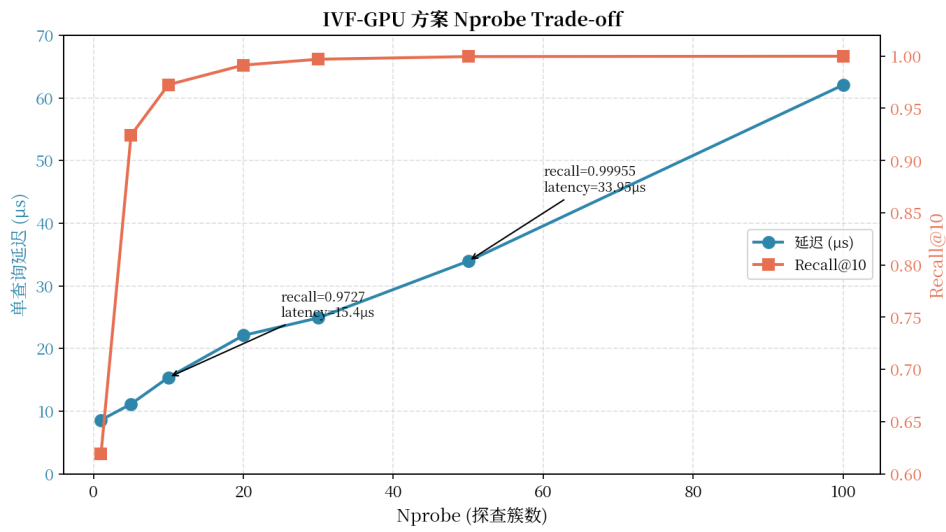
从图 5 可以看出，随着 batch size 增大，单查询延迟单调下降，recall 单调上升。这是因为更大的 batch 意味着 GPU 上同时处理的 query 更多，并行度更高，GPU 利用率更充分；同时更大的 batch 也让 top-k 归约的并行度更高，单查询分摊的归约开销更小。batch size 从 100 增至 1000 时，延迟从 111.9 μs 降至 86.0 μs ，recall 从 0.96 提升至 0.99。这表明在 GPU 检索系统中，尽量增大 batch size 是提升吞吐的有效手段，但受限于显存容量和实时性要求（batch 越大，单次查询等待时间越长）。

(四) IVF Nprobe Trade-off

在 IVF-GPU 方案下，固定 batch size = 200，调整 n_{probe} ，观察 recall 和延迟的变化，结果如表 5 和图 6 所示。

表 5: IVF-GPU 方案下 n_{probe} 的 Trade-off

n_{probe}	recall@10	单查询延迟 (μs)
1	0.619151	8.60
5	0.924303	11.15
10	0.972652	15.40
20	0.991401	22.10
30	0.997000	24.90
50	0.999550	33.95
100	0.999900	62.05

图 6: IVF-GPU 方案下 n_{probe} 对 recall 与延迟的影响

从图 6 可以看出, n_{probe} 与 recall 呈正相关、与延迟也呈正相关。 $n_{\text{probe}} = 1$ 时 recall 仅 0.619, 延迟 $8.6 \mu\text{s}$; $n_{\text{probe}} = 10$ 时 recall 达 0.973, 延迟 $15.4 \mu\text{s}$; $n_{\text{probe}} = 100$ (即全量搜索) 时 recall 接近 1.0, 延迟 $62.05 \mu\text{s}$ 。

(五) IVF 分组化策略对比

将 IVF 基线 (无分组) 与 IVF 分组化策略在不同 n_{probe} 下的延迟进行对比, 结果如表 6 和图 7 所示。

表 6: IVF 基线 vs 分组化策略延迟对比

n_{probe}	IVF 基线延迟 (μs)	分组化延迟 (μs)	recall
1	8.60	125.70	0.619151
10	15.40	337.75	0.972702
20	22.10	418.25	0.991451
30	24.90	438.20	0.997100
50	33.95	540.50	0.999650
100	62.05	522.60	1.000000

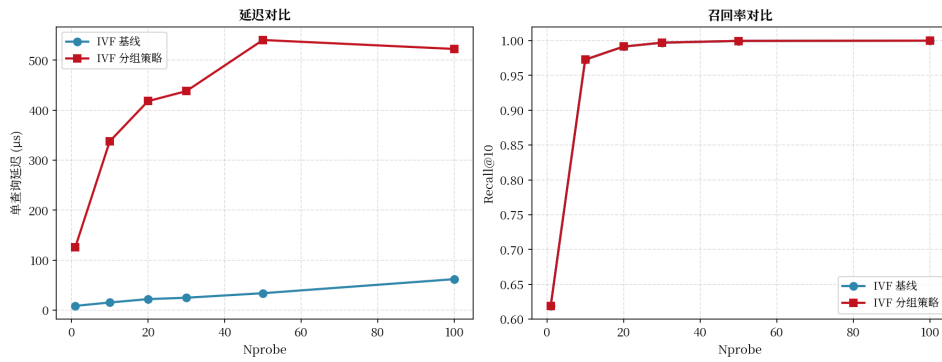


图 7: IVF 基线 vs 分组化策略延迟对比

从图 7 可以看出，分组化策略在本实验数据规模下延迟反而显著高于基线， $n_{\text{probe}} = 10$ 时基线 $15.4 \mu\text{s}$ ，分组化 $337.75 \mu\text{s}$ ，慢了约 22 倍。分析原因：一是本实验 batch size 较小 (200)，分组后每组 query 数量更少，GPU 利用率反而下降；二是分组引入的 CPU 端排序、Kernel 启动开销和显存分配开销过大，远超分组带来的计算节省；三是分组后每组独立启动 Kernel，破坏了原本 batch 级别的矩阵乘法并行度。这一负面结果表明，分组化策略更适合大 batch、高 n_{probe} 的场景，且需要更精细的实现（如将分组逻辑也放到 GPU 上）才能发挥优势。

(六) Recall-Latency 分析

将所有方案在不同参数下的 (recall, 延迟) 数据点绘制在同一散点图中，如图 8 所示。理想方案应位于左上角（高 recall、低延迟）。

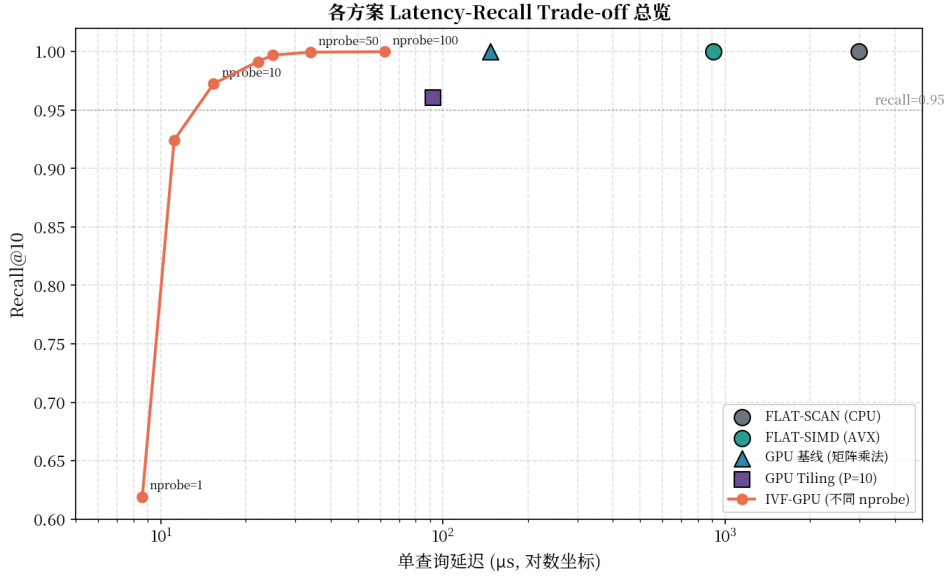


图 8: 各方案 recall-延迟散点图

从图 8 可以看出, IVF-GPU 系列方案整体位于 Pareto 前沿, 即在相同 recall 下延迟最低。GPU Tiling 方案在 recall 0.96 附近延迟约 92 μs 。CPU 方案 (FLAT-SCAN、FLAT-SIMD) 位于右上方, 延迟高但 recall 接近 1.0。这一结果验证了 IVF 索引在 GPU 上的优越性: 通过牺牲少量精度, 可以换取超过 10 倍的延迟降低。

六、 总结

本实验围绕 ANN 选题, 在 Ubuntu 22.04 LTS + RTX 4060 Laptop GPU 环境下, 依次实现并对比了 GPU 矩阵乘法基线、Tiling 优化与 Top-p 归约、IVF 索引加速以及 IVF 分组化策略四种方案, 主要收获如下:

第一, 验证了 GPU 在批量向量检索场景下的显著优势。GPU 基线方案相对 CPU 串行 FLAT-SCAN 实现了约 20 倍加速, IVF-GPU 在 $n_{\text{probe}} = 10$ 时加速比超过 190 倍。

第二, 掌握了 GPU Kernel 优化的核心手段。Tiling 通过共享内存分块减少了全局内存重复访存, Top-p 通过让每个线程维护 P 个候选减少了归约轮数。实验表明 $P = 10$ 是最优取值, 延迟降至 92.05 μs 。通过 cudaEvent 剖析发现, P 的影响集中在 top- k Kernel, matmul Kernel 耗时基本不受影响。

第三, 实现了基于 IVF 倒排索引的近似搜索方案, 通过 K-Means 聚类将全量搜索转化为子空间搜索。

第四, 尝试了 IVF 分组化策略以减少不同 query 访问不同簇导致的并行度浪费, 但实验表明该策略在本实验数据规模下延迟反而显著增加。这一负面结果揭示了 GPU 编程中“并行度提升”与“控制/显存开销增加”之间需要谨慎权衡, 为后续优化指明了方向。